



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

ANISH NAIR

BATCH: A1

PRN:25070122037

Subject: Programming with Java

ASSIGNMENT No : 9

TITLE :Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.

Problem Statement

Write a Java program to demonstrate the use of the final keyword with variables, methods, and classes.

Learning Objectives

To implement the final keyword with variables, methods, and classes.

Theory

The final keyword restricts modification in Java programs. A final variable cannot be reassigned after initialization. A final method cannot be overridden by subclasses, while a final class cannot be inherited. This improves security, prevents accidental modifications, and increases program reliability. Final is commonly used for constants and immutable classes.

Outcome

```

1 interface Printable {
2     void printDetails();
3 }
4 class Student implements Printable {
5     private String name;
6     private int rollNo;
7     public Student(String name, int rollNo) {
8         this.name = name;
9         this.rollNo = rollNo;
10    }
11    @Override
12    public void printDetails() {
13        System.out.println("Student Details -> Name: " + name + " | Roll No: " + rollNo);
14    }
15 }
16 class Employee implements Printable {
17     private String name;
18     private int empId;
19     public Employee(String name, int empId) {
20         this.name = name;
21         this.empId = empId;
22    }
23    @Override
24    public void printDetails() {
25        System.out.println("Employee Details -> Name: " + name + " | Employee ID: " + empId);
26    }
27 }
28 public class MainPrintable {
29     Run main | Debug main
30     public static void main(String[] args) {
31         Printable student = new Student("Alice", 101);
32         Printable employee = new Employee("Bob", 5001);
33
34         student.printDetails();
35         employee.printDetails();
36     }
37 }

```

Listening on 60068

User program running

Student Details -> Name: Alice | Roll No: 101

Employee Details -> Name: Bob | Employee ID: 5001

User program finished

```

1  interface Switchable {
2      |   void turnOn();
3  }
4  class Light implements Switchable {
5      |   @Override
6      |   public void turnOn() {
7      |       |   System.out.println("Light is now turned ON. Brightness set to 100%.");
8      |       }
9  }
10 class Fan implements Switchable {
11     |   @Override
12     |   public void turnOn() {
13     |       |   System.out.println("Fan is now turned ON. Spinning at medium speed.");
14     |       }
15 }
16 public class MainSwitchable {
17     |   Run main | Debug main
18     |   public static void main(String[] args) {
19     |       |   Switchable light = new Light();
20     |       |   Switchable fan = new Fan();
21     |       |
22     |       |   light.turnOn();
23     |       |   fan.turnOn();
24     |   }
25 }

```

```

Listening on 59176
User program running
User program finished
Light is now turned ON. Brightness set to 100%.
Fan is now turned ON. Spinning at medium speed.

```

Conclusion: This Java program successfully demonstrates the practical implementation of interfaces to achieve 100% abstraction and modular design. By defining contracts such as Printable and Switchable, completely unrelated classes—such as Student, Employee, Light, and Fan—can implement common behaviors through their own independent logic.

Exercise

1. Create a **Bank Account program** where account number is final and cannot be changed.
2. Create a **Library Book Management** program where the **book ISBN** is declared as **final** and cannot be changed once assigned. Display the book's ISBN, title, author, and price.